

0.8. Tout l'alphabet, avec deux boucles

La ligne est pleine après T : une seconde boucle écrit U à Z sur la ligne d'en dessous.

1. Ce qu'il faut comprendre

Il y a **26 lettres** et seulement **20 colonnes**. La première boucle écrit A à T sur la ligne 0 ; la seconde écrit les 6 dernières, U à Z, sur la ligne 1.

Dans la seconde boucle, `i` va de 20 à 25 : c'est le bon **indice** dans `ALPHABET` (U est à l'indice 20). Mais la **colonne** doit repartir à 0 : on écrit donc `i - 20`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1399 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

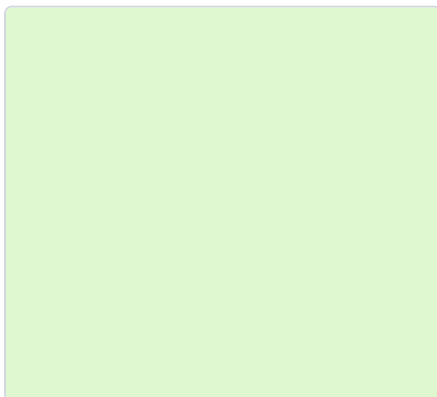
```
int main() {
    for (uint8_t i = 0; i < 20; i++) {    // A à T : ligne 0, colonnes 0 à 19
        poser(i, 0, ALPHABET[i]);
    }

    for (uint8_t i = 20; i < 26; i++) {    // U à Z : ligne 1
        poser(i - 20, 1, ALPHABET[i]);    // la colonne repart à 0
    }

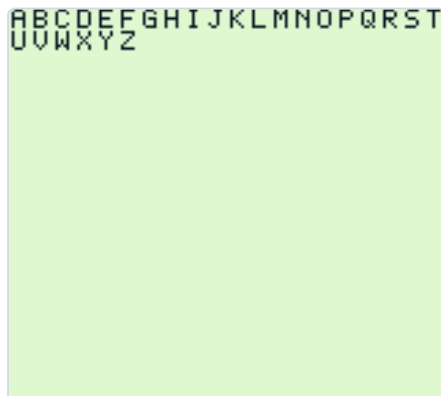
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.8.1. Tout l'alphabet, avec deux boucles — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++